Project 2 Final Report

Slot Machine Game

Ella Stephani and Carissa Lee

UVM Department of Electrical Engineering

EE2185: Circuits Design Project

Instructor: Prof. Tony Barsic

May 5, 2025

Contents

Project Background **2**
- 0.1 Introduction . . . 2
- 0.2 Project Requirements . . . 2
- 0.3 Components . . . 2
  - 0.3.1 Joystick . . . 2
  - 0.3.2 RAGB LEDs . . . 3
  - 0.3.3 Adafruit Metro Mini . . . 5
  - 0.3.4 Shift Registers . . . 6

1 Circuit **7**

2 Code **11**

3 PCB **15**
- 3.1 Troubleshooting . . . 15

4 Testing **19**
- 4.1 Breadboard Performance Matching . . . 19
- 4.2 User Interface . . . 20
- 4.3 Project Requirements . . . 20

5 The Ethics of Gambling in the Modern World **21**

6 Conclusion **21**

7 Sources **22**

8 Appendix **23**

Project Background

Introduction

This slot machine project aims to create a simplified and slightly remodeled design of the classic casino game. The goal is to create a stimulating, interactive game in which a user can pull a mechanical lever that commences the spinning of the slot machine until one by one, each of the three columns lands on a specific light pattern. A win/lose interface will then inform the user of a clear win or loss through a bright flashing LED. This project will utilize an assortment of RAGB LEDs, shift registers, and an Adafruit Metro Mini to format the hardware components. A large portion of this project is expected to be software based.

Project Requirements

- The hardware shall incorporate a physical user activation interface that emulates the tactile nature of the traditional slot machine lever mechanism.

- The hardware shall include a display and the software shall appear to ”spin” the lights to emulate the traditional slot machine game.

- The system shall declare an unambiguous indication of the outcome of the game to the user at the conclusion of each game turn.

Components

This project uses several components to meet its requirements: a joystick, 10 RAGB LEDs, a microcontroller, and 4 shift registers.

0.3.1 Joystick

In order to meet the first requirement of the physical lever mechanism, a joystick is employed. The joystick is an ideal solution in comparison to other options, such as a push button with an attached mechanical arm, because it is found in lab and still meets the requirements.

The joystick used in this project is the Joystick B103 348 (Figure 1). For this project, only one of the joystick’s axes is necessary to act as the lever, and the x-axis is chosen for no particular reason.

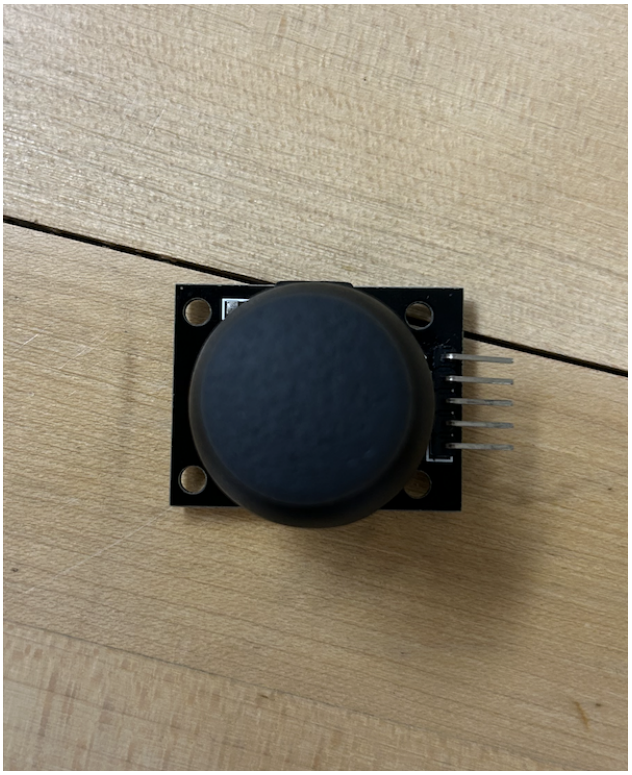


Figure 1: Joystick

0.3.2 RAGB LEDs

To meet the requirement of the spinning effect, the LEDs used will be RAGB. RAGB stands for red, anode, green, blue, which is the order of the pins of the LEDs. When power is connected to the red pin, a red LED turns on, etc. One important feature of RAGB LEDs is their ability to mix their default diode colors. For example if the red pin and blue pin are powered at the same time, the LED appears purple. To test the LEDs, 3.3 volts are sent into each pin, and it is determined all diodes can handle the voltage the device will be sending into them.

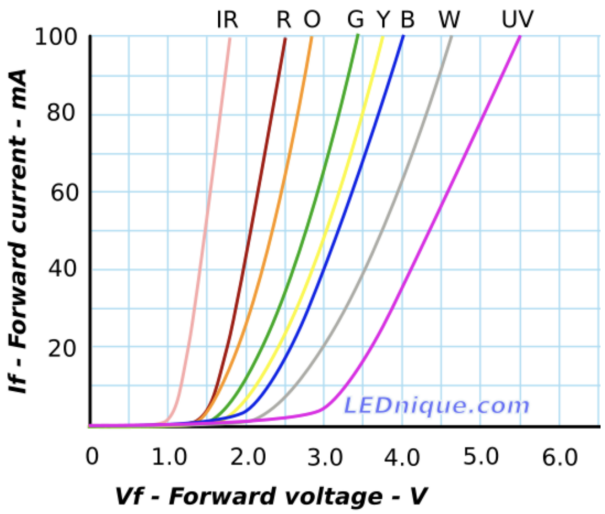


Figure 2: RAGB LED IV Curves [4]

In order to meet the win/lose interface requirements, one large RAGB LED is implemented with a red color designating a loss, and a flashing green light designating a win. Alongside this, nine smaller RAGB LEDs are used in a three by three grid shape to emulate the tradition slot machine display and satisfy the display requirement. Figure 2 below shows a size comparison between the large and small diodes.



Figure 3: RAGB LEDs

0.3.3 Adafruit Metro Mini

The microcontroller utilized in this project is the Adafruit Metro Mini. This computer is responsible for interfacing the software and hardware components. In that, written code that scripts the slot machine display portion is uploaded onto the Metro Mini which coordinates this onto the circuit board created for validation. Figure 4 below shows the Adafruit Metro Mini, and Figure 5 displays its coordinating pin-out schematic.

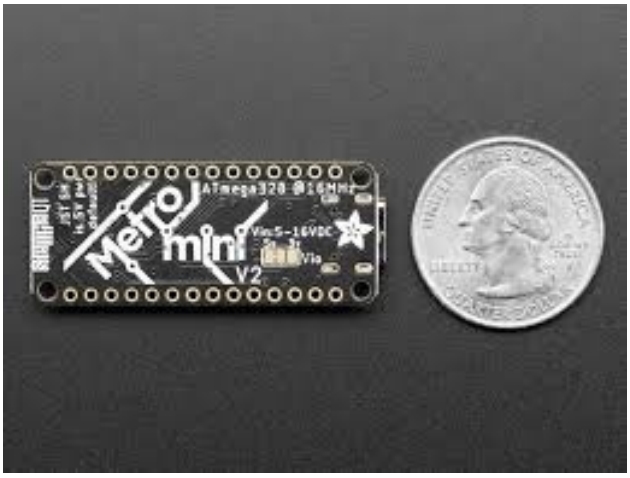


Figure 4: Adafruit Metro Mini [1]

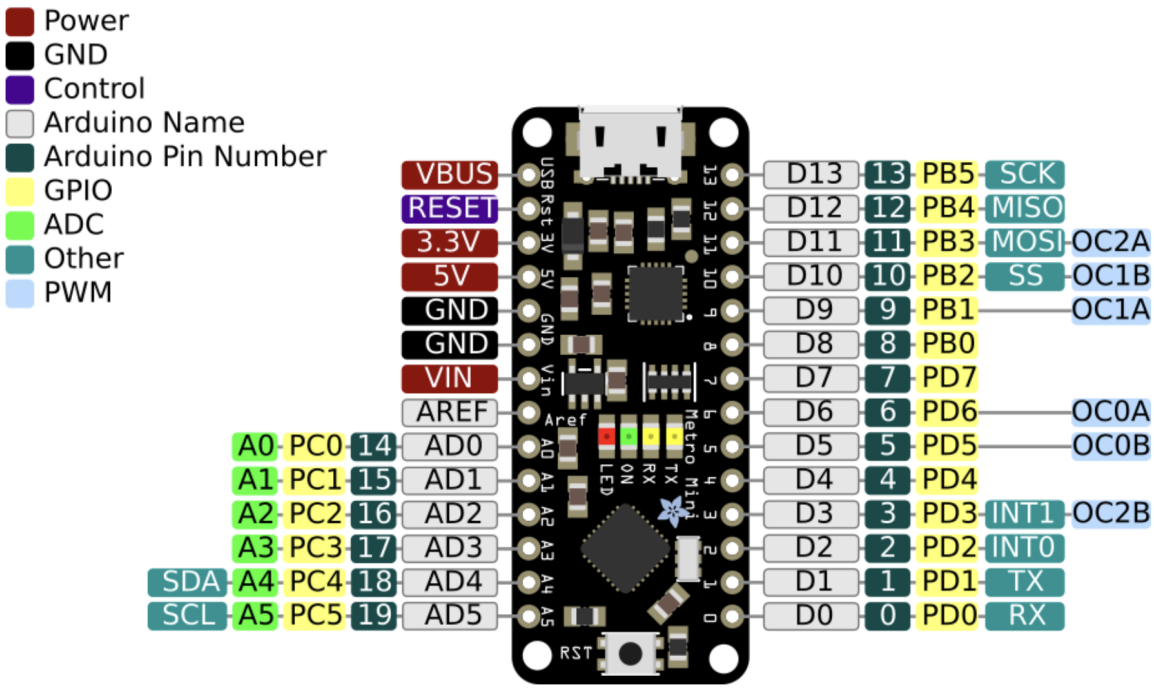


Figure 5: Adafruit Metro Mini Pin-Out [1]

0.3.4 Shift Registers

One of the first problems run into in this project is the number of pins on the Metro Mini. The Metro Mini has 20 input/output pins, but the project requires 10 RAGB LEDs, each with 3 input pins that need to be controlled. Thus, the microcontroller is 10 pins short even before accounting for the joystick. One way to design around this would be to choose a larger controller, but a more unique solution is apparent: shift registers.

A shift register is a type of digital circuit using a cascade of flip-flops where the output of one flip-flop is connected to the input of the next (2). Shift registers can be utilized for bit conversion to expand the number of output pins into the desired amount. The shift register reads in an 8-bit data sequence and shifts it out using specified clock-time intervals to control many outputs with only a few digital I/O pins on the Metro Mini. The pin-out and image of it can be seen below in Figure 6. The shift register used for this project is the 74HC595.

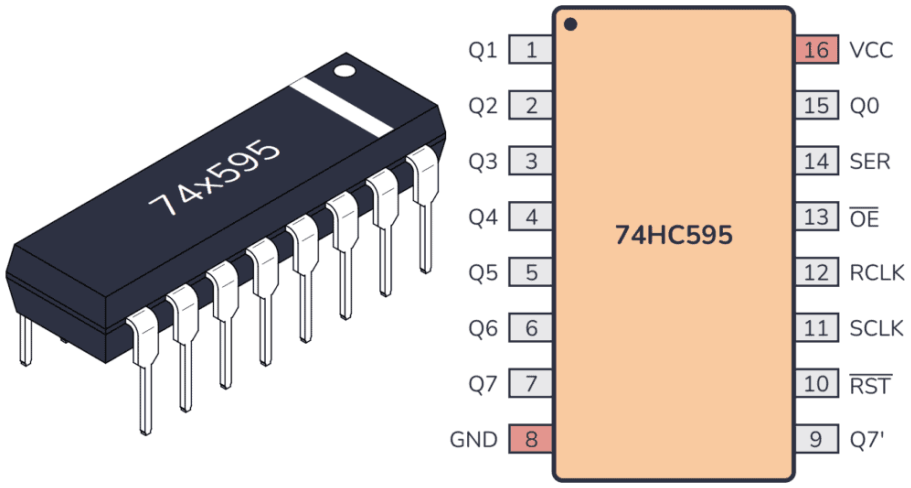


Figure 6: Shift Register and Pin-Out [2]

Circuit

The circuit creation process is more complex than initially expected. As discussed earlier, the first problem addressed is the 10 RGB LEDs, each with three output pins, making that 30 ouput pins. The Metro Mini microcontroller only has 20. To combat this, shift registers are used to expand the number of available digital outputs. More specifically, four 74HC595 are chained together to allow for serial communication from the microcontroller to control eight total output pins from three Metromini output pins.

A rough circuit sketch can be seen below in Figure 7. This step initializes the creation process and many ideas are thrown around regarding design elements that could truly emulate the slot machines traditional nature.

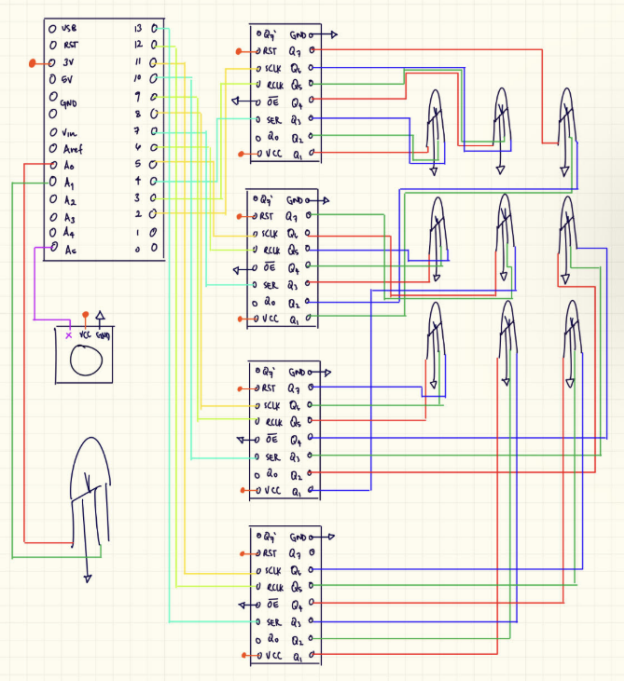


Figure 7: Circuit Schematic

The next steps include physically creating this board on a breadboard which can be seen below in Figure 8.

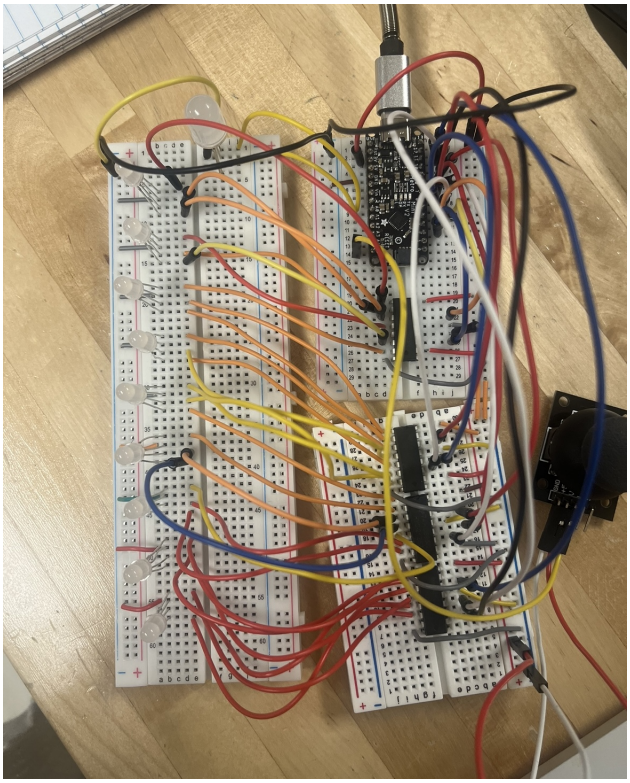


Figure 8: Circuit Created on Breadboard

Initial testing is conducted by supplying the LEDs with a voltage supply directly to ensure that all lights within the diodes are working. This can be seen below in Figure 9.

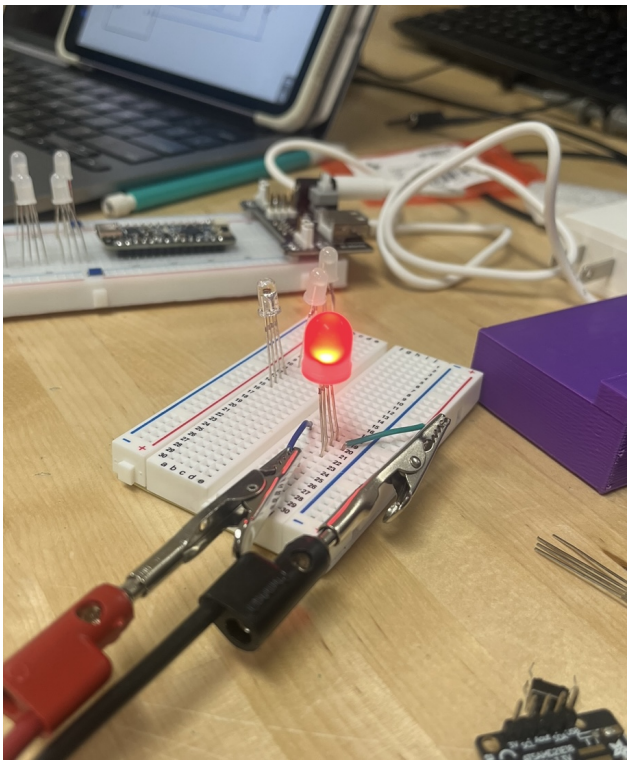


Figure 9: LED Testing

Alongside this, it is observed that each of the diodes are able to intake 3.3V without causing damage to the component, so it is decided that 3.3V would power the circuit as to directly supply power without the use of resistors. This cuts down on the complexity of the circuit, and the total numbers of components.

Initial code testing involves figuring out how to code for the shift registers (more can be seen in the Code section below). Once the semantics are figured out, the circuit is tested by turning on all of the LEDs. An image of some of the breadboard testing can be seen below in Figure 10.

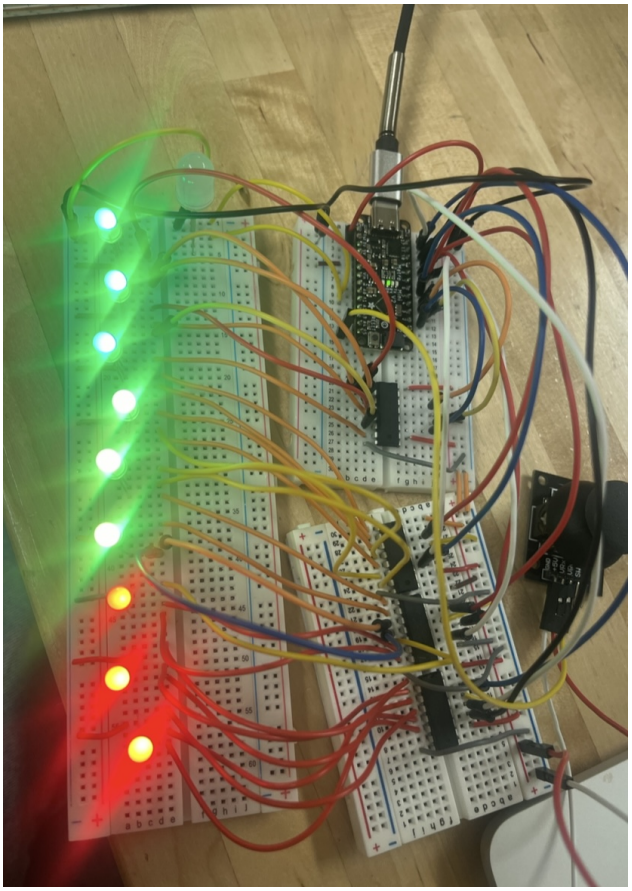


Figure 10: Working Circuit

As seen in Figure 10 above, the wired circuit proves successful as the lights are grouped into sets of three, as consistent with the design, each showing the same color.

After building this complex circuit, it is clear to the designers that shift registers have very useful properties. The ability to control so many outputs with such a small controller just by writing bitstreams is a powerful tool. By working with shift registers so thoroughly, the designers will never forget their utility. On the other hand, these components do have some drawbacks. The wiring is intense and can be tedious at times. Troubleshooting such a complex circuit can be laborious. Tradeoffs are inevitable, but the final circuit is a unique solution to a curious problem.

Code

The code is a large part of this project as it dictates the behavior of every LED, and the full code can be seen in the appendix. The designer acknowledges the inefficiencies in the code but kindly declines to, for lack of better words, fix something that is not broken.

The first step of the game is to pull the lever and initiate the code's loop. To achieve this, the joystick's x-axis value is measured, and when it is far to one side, the intro scroll begins.

```
// when the joystick is pushed, the scroll begins
if (!scrollStarted && (joystick_val > 700)) {
    scrollStarted = true;
  // small delay
  delay(300);
```

To change the color of the LEDs, data first has to be sent into the shift registers [3]. This is done by lowering the latch, then sending in the bitwise data corresponding to the desired colors for all LEDs in the 3x3 display, and finally closing the latch pins. A delay to assure the colors are visible is included at the end. An example can be seen in the code snippet below:

```
    digitalWrite(latchPin0, LOW);
    digitalWrite(latchPin1, LOW);
    digitalWrite(latchPin2, LOW);
    digitalWrite(latchPin3, LOW);
    // row 1 = red
    // row 2 = purple
    // row 3 = blue
    shiftOut(dataPin0, clockPin0, MSBFIRST, 0b10010010);
    shiftOut(dataPin1, clockPin1, MSBFIRST, 0b01101000);
    shiftOut(dataPin2, clockPin2, MSBFIRST, 0b10010110);
    shiftOut(dataPin3, clockPin3, MSBFIRST, 0b01001000);
    // all latch pins = HIGH to send the data
    digitalWrite(latchPin0, HIGH);
    digitalWrite(latchPin1, HIGH);
    digitalWrite(latchPin2, HIGH);
    digitalWrite(latchPin3, HIGH);
    // delay to keep the colors for 100 ms
    delay(100);
```

The bitstreams within the shiftOut command correspond to the pins of the LEDs. It is convoluted due to certain LEDs being split across multiple resistors, so tables will be used to help demonstrate. The bits in the shiftOut command are formatted as 0b76543210. The shift register pinout sheet contains the names of the pins referenced in Figure 6. The convention for the corresponding LED pins in the tables will be R (row number) C (column number) : LED (LED number) pin color.

Table 1: Shift Register 0 Bits

Shift Register Pin Name	Bit Number	Corresponding LED Pin
Q1	0	unused
Q2	1	R1C1: LED1 red
Q3	2	R1C1: LED1 green
Q4	3	R1C1: LED1 blue
Q5	4	R1C2: LED2 red
Q6	5	R1C2: LED2 green
Q7	6	R1C2: LED2 blue
Q8	7	R1C3: LED3 red

Table 2: Shift Register 1 Bits

Shift Register Pin Name	Bit Number	Corresponding LED Pin
Q1	0	unused
Q2	1	R1C3: LED3 green
Q3	2	R1C3: LED3 blue
Q4	3	R2C1: LED4 red
Q5	4	R2C1: LED4 green
Q6	5	R2C1: LED4 blue
Q7	6	R2C2: LED5 red
Q8	7	R2C2: LED5 green

Table 3: Shift Register 2 Bits

Shift Register Pin Name	Bit Number	Corresponding LED Pin
Q1	0	unused
Q2	1	R2C2: LED5 blue
Q3	2	R2C3: LED6 red
Q4	3	R2C3: LED6 green
Q5	4	R2C3: LED6 blue
Q6	5	R3C1: LED7 red
Q7	6	R3C1: LED7 green
Q8	7	R3C1: LED7 blue

Table 4: Shift Register 3 Bits

Shift Register Pin Name	Bit Number	Corresponding LED Pin
Q1	0	unused
Q2	1	R3C2: LED8 red
Q3	2	R3C2: LED8 green
Q4	3	R3C2: LED8 blue
Q5	4	R3C3: LED9 red
Q6	5	R3C3: LED9 green
Q7	6	R3C3: LED9 blue
Q8	7	unused

In order to make the LEDs appear to ”spin”, the first row begins red, and in the next shift the second row is red, and in the next shift the third row is red, and so on for each color. This creates the illusion that each color is moving down the display.

After the intro scroll, the game begins. The first column is scrolled through similarly to the intro scroll. In order to randomize the color set the scroll lands on, a random break-point is set and compared to after each color change. The colors are also stored in a bitwise variable to keep them on while the next column scrolls. All of these features can be seen in an abridged code snippet below:

```
// setting a random breakpoint between 12 and 50 color changes for the first
  int breakpoint1 = random(12, 50);

  // start with 0 color changes
  int i = 0;
  // making sure while loop cannot go past 51 changes
  while(i<51){

  // col 1 row 1 = yellow
  shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00000110);
  // storing the value of the led by grabbing only the
  three bits that control it
  bitstream0 = 0b00001110 & 0b00000110;

  // checking to see if the random breakpoint is reached
  if (i == breakpoint1){
    break;
  }
  // if not, add 1 to i
  else{
    i++;
  }
```

The final component of the code is to analyze whether the game has been won or lost. The LEDs in the third row are compared to each other, and if all three are the same color, the game is deemed

to be won. The bits corresponding to each of the row 3 LEDs are pulled out from the variable they were initially stored in, and moved into a 3 bit variable such that they can be compared. If the game is won, the large LED flashes green and then holds for 10 seconds. If the game is lost, the LED turns red and holds for 5 seconds. Then, all the LEDs turn off, and the game can be played again. An abridged code snippet can be seen below:

```
// grabbing the bits from all leds in row 3
// moving the led bits over to make them comparable
byte bitsA = (bitstream2 & 0b11100000) >> 5;
byte bitsB = (bitstream33 & 0b00001110) >> 1;
byte bitsC = (bitstream333 & 0b01110000) >> 4;

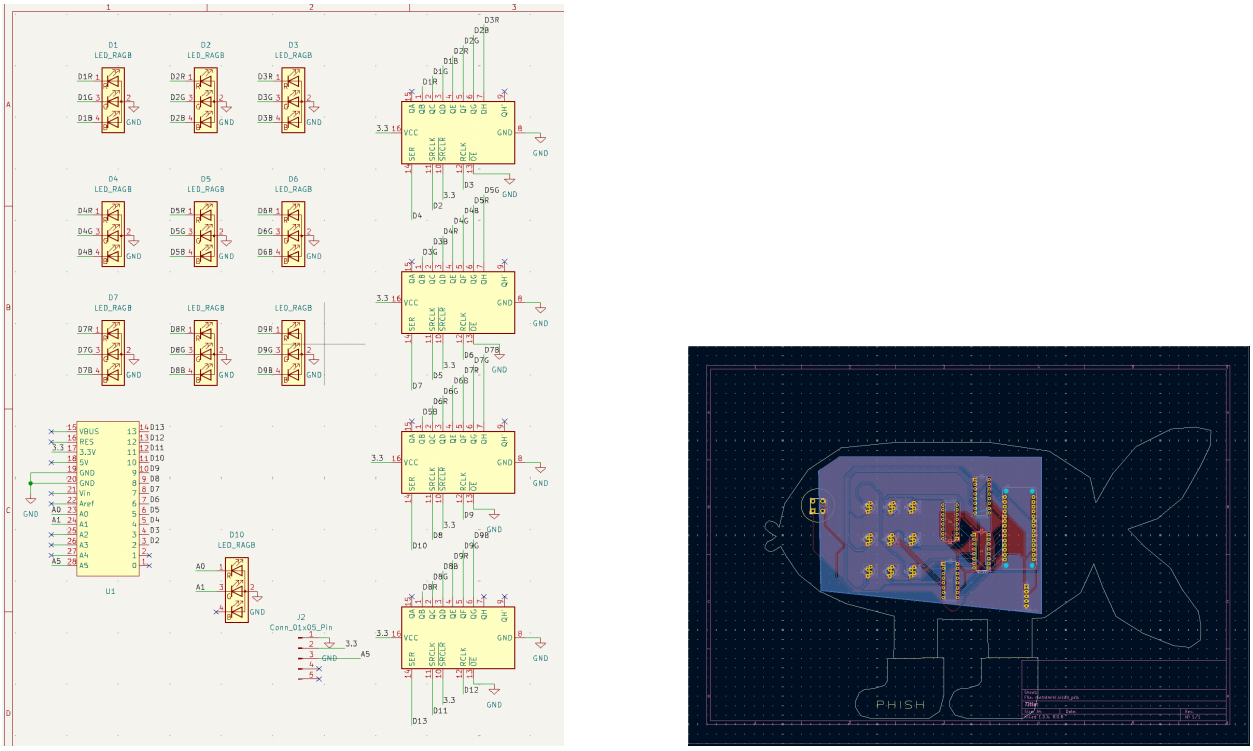
// if all 3 leds are equal
if (bitsA == bitsB && bitsB == bitsC) {
  // blinking green on medium led
  digitalWrite(A1, HIGH);
  delay(200);

// if leds are different
else {
  // medium led turns red
  digitalWrite(A0, HIGH);

// reset game
scrollStarted = false;
```

PCB

The printed circuit board creation is designed with reference to the working breadboard model. Due to the amount of connections needed, nets are used to tidy the schematic. The PCB Layout implements measured footprints and is used to generate DRILL and GERBER files for purchasing. Figure 11 below shows the PCB schematic and layout created on the Kicad software.



(a) PCB Schematic

(b) PCB Design

Figure 11: Kicad Printed Circuit Board Designs

The chosen supplier is JLCPCB. The estimated cost for the board is $15, but due to recent imposed tariffs, there is a $40 tax on it. In combination with shipping, the order costs $90.

As seen in Figure 11, the microcontroller is created with according pin-outs. Both the schematic icon and footprint are handmade using measurements from the board itself.

There was an original error made during the first round of PCB ordering in which the traces were not drawn in the Kicad editor before purchase, so the first board came without any copper traces/-connections and only through holes. This was remedied and the final design is above. The first incorrect board came in four days after it was ordered.

Troubleshooting

Due to longer shipping times with the fixed PCB order, the erroneous board was used and altered. To begin, all componets were placed into their respective spots on the through hole PCB, and they all fit perfectly. This is below in Figure 12.

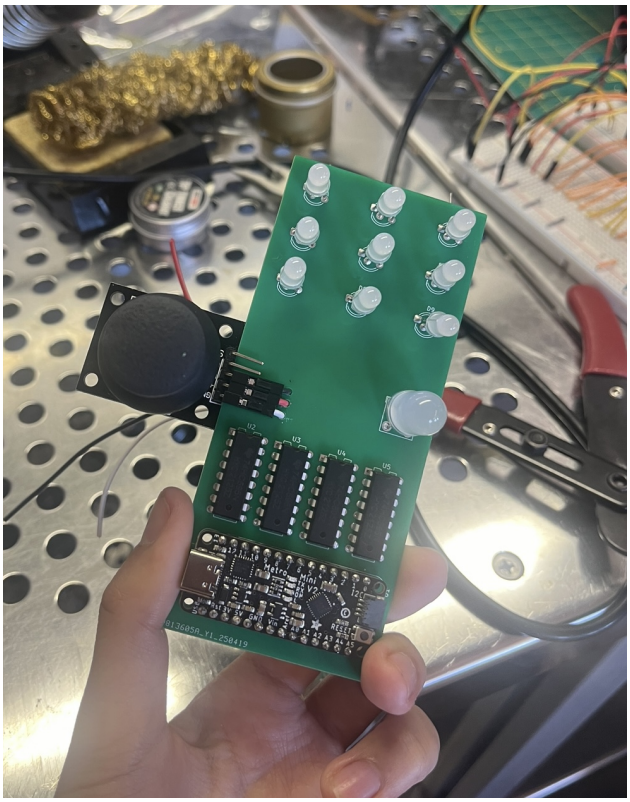


Figure 12: PCB with Components

Initially, it was attempted to solder each individual missing copper trace, but this quickly proved unsuccessful as there were over 71 traces that needed to be routed and conductive parts ended up overcrossing and shorting the circuit. An image of this can be seen below in Figure 13.

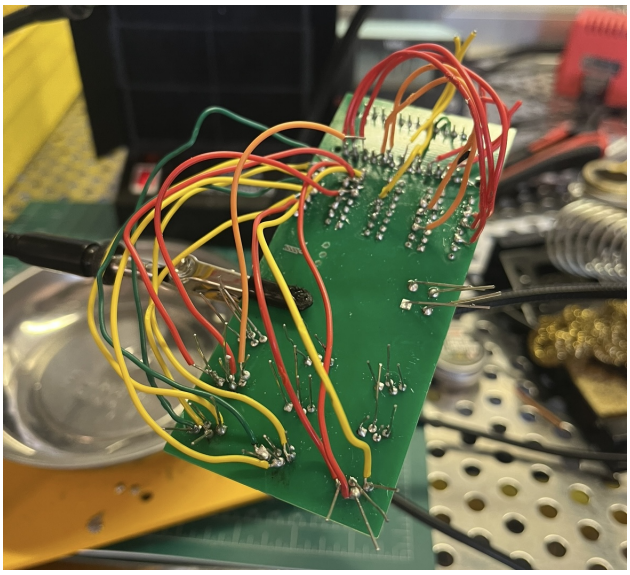


Figure 13: Attempted wire soldered PCB

To combat this, the part of the PCB that held the smaller nine RGB LEDs was cut off from the bottom half that held the other components. These LEDs were then soldered onto the broken PCB and connected with long wires and secured them with heat shrink tubes. Figure 14 below shows the solder job of the final PCB.

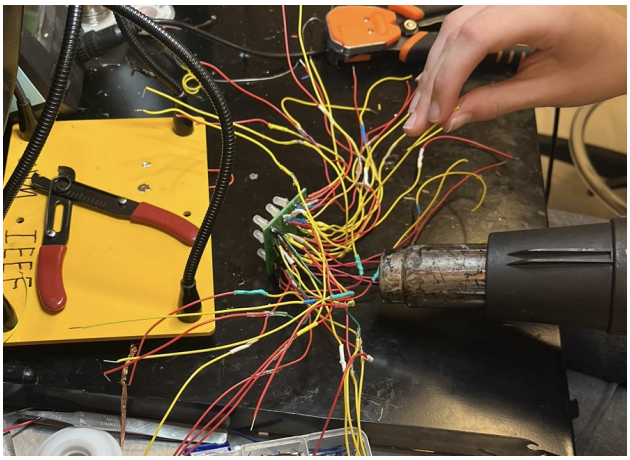


Figure 14: LEDs Soldered onto PCB and Extended with Wires

After this, the other ends of the LEDs, the metromini, and the shift registers were all connected to a breadboard. An image of these connections can be seen below in Figure 15

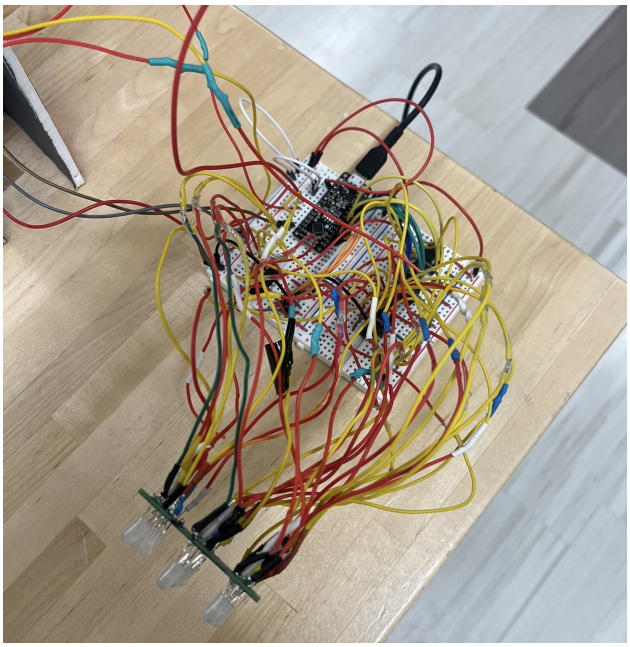


Figure 15: Breadboard to PCB Connections

To support the structural soundness of the wiring and add some creativity, a housing unit was

created from foamboard and paint to make it look like a slot machine. The PCB portion responsible for holding the LEDs was fit snuggly into its housing component to emulate the display of a traditional slot machine. An image of the final result can be seen below in Figure 16.

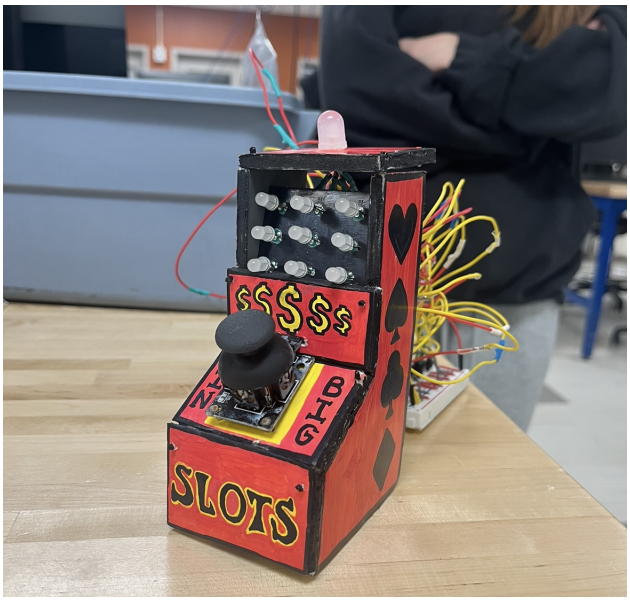


Figure 16: Completed Project in Mechanical Housing

Testing

The first stages of testing included determining if all colors of the RGB LEDs are able to withstand 3.3V without blowing. Follow up hardware testing could include testing how much voltage/current travels through each leg of the LED in order to determine if it is damaging the components.

Breadboard Performance Matching

To ensure than the performance of the PCB-breadboard matrix was working the same as that of when it was originally wires on a breadboard only, voltage current tests were completed to ensure that all solder joints were properly created to conduct electricity. To do this, the DC power supply was used at 3.3V with a current limit of 20 milliamps and connected to each RGB pin one my one of each LED to ensure that they were all working. It was found that there were a few loose solder joints, so this was remedied by using a heatgun and pliers to reconnect the joint mechanically.

As seen below in Figure 17, all LEDs were found to turn on properly to the necessary colors, hence performing in line with what was expected from prior breadboard testing.



Figure 17: Final Design with LEDs On

User Interface

This project is a game-design and its power supply will derived from a computer plug in. Due to this, most relevant testing is more focused on user interface testing. Breadboard user testing has already begun in which users are tasked with attempting to use the circuit without prior instruction. It has proven extremely successful in which all users understand how to play the game, and find it very fun and interactive. To ensure that users understand the game, the following tests were conducted:

- Users will be asked if they know if they won or loss at the conclusion of each game to ensure that the win/lose interface is unambiguous
- It will be observed if users understand how to initiate the game
- Users reactions will be observed. In example, if users reach to immediately play again, it is understood that the game is fun and addicting (like the true nature of gambling!).

The feedback recieved showed positive results and it was determined that users not only innately knew how to use the game, or were able to figure it out quickly, and enjoyed it and had emotional reactions, compelling them to keep playing. All users were able to easily and accurately understand if their game turn ended in a win or a loss based on the signaling light and the display lineup.

Project Requirements

As aforementioned on page 3, there were three major project requirements for this games' success. Specific testing was developed for each of these and conducted on a variety of users. Table 1 below shows the outline of how the project requirements were fulfilled.

Project Requirement	**Implementation**	**User Testing**	**Success?**
The hardware shall incorporate a physical user activation interface that emulates the tactile nature of the traditional slot machine lever mechanism.	Used a joystick with a pull-down mechanism to replicate the lever feel.	Users were asked to interact with the joystick to simulate a game start. Feedback indicated it was intuitive.	Yes
The hardware shall include a display and the software shall appear to "spin" the lights to emulate the traditional slot machine game.	Designed a 3x3 LED matrix display with code to create a spinning animation effect.	Users visually followed the "spinning" lights and understood the slot game emulation.	Yes

The system shall declare an unambiguous indication of the outcome of the game to the user at the conclusion of each game turn.	Bright lights were used to show a red (loss) or green (win) outcome at the end of the game turn.	Users correctly interpreted the game result based on the final color, with no confusion reported.	Yes

The Ethics of Gambling in the Modern World

The definition of gambling: to wager something of value on a *random* event with the intent of winning something of value [7]. From horse racing in the late Han Dynasty in ancient China [5] to casting lots on Jesus' crucifixion outfit [6], gambling has been woven into human history for ages. While it may remain the same at its core- never without risk and repercussion- how has gambling changed throughout time?

Before the age of mobile sports betting- which is a fascinating phenomenon of its own- casinos have been the epitome of the modern gambling world. Casinos are convenient, centralized locations where gamblers can make bets on whatever their hearts desire, yet they have an intentionally sinister design. Casinos purposefully do not implement windows in an attempt to prevent gamblers from noticing the passage of time. They are designed to resemble labyrinths; as a gambler goes to the bathroom or to the exit, the maze-like design tries to slow them down to lure them in for one more game. Arguably most notably, the light displays on the machines are bright and eye catching, the clinking sound of coins is constant, and these sensory elements are designed to dazzle the patrons and keep them engaged as long as possible [8]. The design of these venues illustrates how gambling has been commercialized and strategically manipulated to exploit the average individual. Excitement associated with the action of taking risks emits rushes of dopamine to the brain, regardless of a win or a loss, which is the driving factor behind gambling addictions. Corporations harness this natural human response in order to prey on the general population in the name of a profit, and in the opinion of the writer, they cross quite a troubling ethical line.

Severe gambling addiction affects roughly 2 million Americans, and this addiction has catastrophic consequences. While sports betting is on the rise, slots are currently recognized as the most addictive method of gambling. 80% of addicts will never seek help, and of those who do, 70% will relapse. Gambling addicts are prone to going into extreme debt, and sadly, are 15 times more likely to commit suicide [9]. I implore the reader to reflect on the further implications of these statistics and how they relate to the value of money in a capitalistic society, as well as the predation by corporations as touched on earlier.

It is worth noting that this project is an emulation of a slot machine and is not intended to promote gambling. If anything, the creation of this mini slot machine is a shoe-in to discuss a muddled ethical dilemma.

Conclusion

Despite the many difficulties throughout this process, the end result proved to be fun to play and easy to interact with.

If we were to remake the project, the immediate change would be to order to PCB properly with the correct traces. This would have made the entire assembly process much more simple to save time and also make it easier to follow. Had this step been completed properly, the entire electrical component could have been housed within the mechanical block without wires sticking all about. Alongside this, the PCB would have been more structurally sound than the varying wires connected to wires.

If given more time on this, we would also like to explore the addition of a speaker to emulate the rolling sound of a slot machine and to announce verbally if a round was a win or a lose.

This project gave us insight into using shift registers, microcontrollers, and complex code towards creating a seemingly simple game. Learning how to use shift registers and then be able to store the data and compare was necessary towards creating the rolling display and being able to determine if a round was a win or a lose. This involved a lot of troubleshooting and talking to a variety of professors and gaining new perspective and insights. This project utilized the skills we learned in computer science courses, digital design, and circuits to create this intersectionality of the three. Although there were many missteps taken throughout this project, we are very content with the outcome and are happy we created something they other people enjoy as well.

Sources

[1] Ada, Lady. "Adafruit Metro Mini." *Adafruit Learning System*, 13 Aug. 2017, learn.adafruit.com/adafruit-metro-mini/pinouts.

[2] "Shift Register." *Wikipedia*, Wikimedia Foundation, 27 Apr. 2025, en.wikipedia.org/wiki/Shift_register.

[3] Maw, Carlyn, and Tom Igoe. "Serial to Parallel Shifting-Out with a 74HC595." *Docs.Arduino.Cc*, 25 Jan. 2022, docs.arduino.cc/tutorials/communication/guide-to-shift-out/.

[4] "IV Curves." *LEDnique*, 29 Nov. 2017, lednique.com/current-voltage-relationships/iv-curves/.

[5] Jiahui, Sun. "Dice to Dominoes: Gambling in Ancient China." *The World of Chinese*, 5 May 2023, www.theworldofchinese.com/2023/05/dice-dominoes-and-deadly-games-of-chance-gambling-in-ancient-china/.

[6] Glimne, Dan. "Casino." *Encyclopædia Britannica*, Encyclopædia Britannica, inc., 15 Apr. 2025, www.britannica.com/topic/casino-gambling-house.

[7] "Gambling." *Wikipedia*, Wikimedia Foundation, 17 Apr. 2025, en.wikipedia.org/wiki/Gambling.

[8] Keaton, Brooke. "Casino Design: The Sneaky Tricks That Make You Spend More." *Casino.Org*, 21 July 2020, www.casino.org/blog/casino-design-psychology/.

[9] Help, Written by Addiction, et al. "Gambling Addiction - Gambling Prevalence & Demographics (2025)." *AddictionHelp.Com*, 1 Jan. 2025, www.addictionhelp.com/gambling/statistics/.

Appendix

Full commented code:

```
//-------------------------------------------------------------
//-------------------------------------------------------------


// defining variables


//-------------------------------------------------------------
//-------------------------------------------------------------

// defining the bytes where the led bits will be stored once
// they are landed on
// allows us to keep one column stable while scrolling the next column
// also allows us to check the color of the leds at the end of the game
byte bitstream0;
byte bitstream1;
byte bitstream2;
byte bitstream3;
byte bitstream00;
byte bitstream11;
byte bitstream22;
byte bitstream33;
byte bitstream000;
byte bitstream111;
byte bitstream222;
byte bitstream333;

// defining pins

// shift register 0
int clockPin0 = 2;
int latchPin0 = 3;
int dataPin0 = 4;
// shift register 1
int clockPin1 = 5;
int latchPin1 = 6;
int dataPin1 = 7;
// shift register 2
int clockPin2 = 8;
int latchPin2 = 9;
int dataPin2 = 10;
```

```
// shift register 3
int clockPin3 = 11;
int latchPin3 = 12;
int dataPin3 = 13;

// scrolling condition
bool scrollStarted = false;

void setup(){
// setting pinmodes for shift registers
pinMode(clockPin0, OUTPUT);
pinMode(latchPin0, OUTPUT);
pinMode(dataPin0, OUTPUT);
pinMode(clockPin1, OUTPUT);
pinMode(latchPin1, OUTPUT);
pinMode(dataPin1, OUTPUT);
pinMode(clockPin2, OUTPUT);
pinMode(latchPin2, OUTPUT);
pinMode(dataPin2, OUTPUT);
pinMode(clockPin3, OUTPUT);
pinMode(latchPin3, OUTPUT);
pinMode(dataPin3, OUTPUT);

// setting pinmodes for medium LED
pinMode(A0, OUTPUT);
pinMode(A1, OUTPUT);

// setting pinmode for joystick
pinMode(A5, INPUT);

// setting all LEDs off
digitalWrite(latchPin0, LOW);
shiftOut(dataPin0, clockPin0, LSBFIRST, 0);
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, LOW);
shiftOut(dataPin1, clockPin1, LSBFIRST, 0);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, LOW);
shiftOut(dataPin2, clockPin2, LSBFIRST, 0);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, LOW);
shiftOut(dataPin3, clockPin3, LSBFIRST, 0);
digitalWrite(latchPin3, HIGH);

// unused pin for a random seed
```

```
randomSeed(analogRead(A2));

// serial monitor for debugging
Serial.begin(9600);
}



//-------------------------------------------------------
//-------------------------------------------------------


void loop() {

  // setting LEDs off
  digitalWrite(latchPin0, LOW);
  shiftOut(dataPin0, clockPin0, LSBFIRST, 0);
  digitalWrite(latchPin0, HIGH);
  digitalWrite(latchPin1, LOW);
  shiftOut(dataPin1, clockPin1, LSBFIRST, 0);
  digitalWrite(latchPin1, HIGH);
  digitalWrite(latchPin2, LOW);
  shiftOut(dataPin2, clockPin2, LSBFIRST, 0);
  digitalWrite(latchPin2, HIGH);
  digitalWrite(latchPin3, LOW);
  shiftOut(dataPin3, clockPin3, LSBFIRST, 0);
  digitalWrite(latchPin3, HIGH);

  // defining joystick value
  int joystick_val = analogRead(A5);


//-------------------------------------------------------
//-------------------------------------------------------


// intro scroll


//-------------------------------------------------------
//-------------------------------------------------------



// when the joystick is pushed, the scroll begins
```

```
if (!scrollStarted && (joystick_val > 700)) {
    scrollStarted = true;
  // small delay
  delay(300);
  // scrolling through this loop 8 times
  for (int i = 0; i < 8; i++){
    // all latch pins = LOW to set the data
    digitalWrite(latchPin0, LOW);
    digitalWrite(latchPin1, LOW);
    digitalWrite(latchPin2, LOW);
    digitalWrite(latchPin3, LOW);
    // row 1 = red
    // row 2 = purple
    // row 3 = blue
    shiftOut(dataPin0, clockPin0, MSBFIRST, 0b10010010);
    shiftOut(dataPin1, clockPin1, MSBFIRST, 0b01101000);
    shiftOut(dataPin2, clockPin2, MSBFIRST, 0b10010110);
    shiftOut(dataPin3, clockPin3, MSBFIRST, 0b01001000);
    // all latch pins = HIGH to send the data
    digitalWrite(latchPin0, HIGH);
    digitalWrite(latchPin1, HIGH);
    digitalWrite(latchPin2, HIGH);
    digitalWrite(latchPin3, HIGH);
    // delay to keep the colors for 100 ms
    delay(100);

    // all latch pins = LOW to set the data
    digitalWrite(latchPin0, LOW);
    digitalWrite(latchPin1, LOW);
    digitalWrite(latchPin2, LOW);
    digitalWrite(latchPin3, LOW);
    // row 1 = yellow
    // row 2 = red
    // row 3 = purple
    shiftOut(dataPin0, clockPin0, MSBFIRST, 0b10110110);
    shiftOut(dataPin1, clockPin1, MSBFIRST, 0b01001010);
    shiftOut(dataPin2, clockPin2, MSBFIRST, 0b10100100);
    shiftOut(dataPin3, clockPin3, MSBFIRST, 0b01011010);
    // all latch pins = HIGH to send the data
    digitalWrite(latchPin0, HIGH);
    digitalWrite(latchPin1, HIGH);
    digitalWrite(latchPin2, HIGH);
    digitalWrite(latchPin3, HIGH);
    // delay to keep the colors for 100 ms
    delay(100);
```

```
// all latch pins = LOW to set the data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);
// row 1 = green
// row 2 = yellow
// row 3 = red
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00100100);
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b11011010);
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b00101100);
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00010010);
// all latch pins = HIGH to send the data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// delay to keep the colors for 100 ms
delay(100);

// all latch pins = LOW to set the data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);
// row 1 = blue-green
// row 2 = green
// row 3 = yellow
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b01101100);
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b10010110);
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b01101000);
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00110110);
// all latch pins = HIGH to send the data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// delay to keep the colors for 100 ms
delay(100);

// all latch pins = LOW to set the data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
```

```
    digitalWrite(latchPin3, LOW);
    // row 1 = blue
    // row 2 = blue-green
    // row 3 = green
    shiftOut(dataPin0, clockPin0, MSBFIRST, 0b01001000);
    shiftOut(dataPin1, clockPin1, MSBFIRST, 0b10110100);
    shiftOut(dataPin2, clockPin2, MSBFIRST, 0b01011010);
    shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00100100);
    // all latch pins = HIGH to send the data
    digitalWrite(latchPin0, HIGH);
    digitalWrite(latchPin1, HIGH);
    digitalWrite(latchPin2, HIGH);
    digitalWrite(latchPin3, HIGH);
    // delay to keep the colors for 100 ms
    delay(100);

    // all latch pins = LOW to set the data
    digitalWrite(latchPin0, LOW);
    digitalWrite(latchPin1, LOW);
    digitalWrite(latchPin2, LOW);
    digitalWrite(latchPin3, LOW);
    // row 1 = purple
    // row 2 = blue
    // row 3 = blue-green
    shiftOut(dataPin0, clockPin0, MSBFIRST, 0b11011010);
    shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00100100);
    shiftOut(dataPin2, clockPin2, MSBFIRST, 0b11010010);
    shiftOut(dataPin3, clockPin3, MSBFIRST, 0b01101100);
    // all latch pins = HIGH to send the data
    digitalWrite(latchPin0, HIGH);
    digitalWrite(latchPin1, HIGH);
    digitalWrite(latchPin2, HIGH);
    digitalWrite(latchPin3, HIGH);
    // delay to keep the colors for 100 ms
    delay(100);
  }


//-------------------------------------------------------
//-------------------------------------------------------


// scrolling first column
```

```
//-------------------------------------------------------
//-------------------------------------------------------


  // setting a random breakpoint between 12 and 50 color changes
  // for the first while loop
  int breakpoint1 = random(12, 50);

  // start with 0 color changes
  int i = 0;
  // making sure while loop cannot go past 51 changes
  while(i<51){
  // latches low to set data
  digitalWrite(latchPin0, LOW);
  digitalWrite(latchPin1, LOW);
  digitalWrite(latchPin2, LOW);
  digitalWrite(latchPin3, LOW);

  // col 1 row 1 = red
  shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00000010);
  // storing the value of the led by grabbing only the three bits
  // that control it
  bitstream0 = 0b00001110 & 0b00000010;

  // col 1 row 2 = purple
  shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00101000);
  // storing the value of the led by grabbing only the three bits
  // that control it
  bitstream1 = 0b00111000 & 0b00101000;

  // col 1 row 3 = blue
  shiftOut(dataPin2, clockPin2, MSBFIRST, 0b10000000);
  // storing the value of the led by grabbing only the three bits
  // that control it
  bitstream2 = 0b11100000 & 0b10000000;

  // third shift register goes unused in this column; but still
  // included for consistency
  shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
  bitstream3 = 0b00000000;

  // latches high to send data
  digitalWrite(latchPin0, HIGH);
  digitalWrite(latchPin1, HIGH);
  digitalWrite(latchPin2, HIGH);
```

```
digitalWrite(latchPin3, HIGH);
// small delay to see colors
delay(100);

// checking to see if the random breakpoint is reached
if (i == breakpoint1){
  break;
}
// if not, add 1 to i
else{
  i++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// col 1 row 1 = yellow
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00000110);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream0 = 0b00001110 & 0b00000110;

// col 1 row 2 = red
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00001000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream1 = 0b00111000 & 0b00001000;

// col 1 row 3 = purple
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b10100000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream2 = 0b11100000 & 0b10100000;

// third shift register goes unused in this column; but still
// included for consistency
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
bitstream3 = 0b00000000;

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
```

```
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// small delay to see colors
delay(100);

// checking to see if the random breakpoint is reached
if (i == breakpoint1){
  break;
}
// if not, add 1 to i
else{
  i++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// col 1 row 1 = green
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00000100);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream0 = 0b00001110 & 0b00000100;

// col 1 row 2 = yellow
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00011000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream1 = 0b00111000 & 0b00011000;

// col 1 row 3 = red
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b00100000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream2 = 0b11100000 & 0b00100000;

// third shift register goes unused in this column; but still
// included for consistency
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
bitstream3 = 0b00000000;

// latches high to send data
digitalWrite(latchPin0, HIGH);
```

```
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// small delay to see colors
delay(100);

// checking to see if the random breakpoint is reached
if (i == breakpoint1){
  break;
}
// if not, add 1 to i
else{
  i++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// col 1 row 1 = blue-green
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00001100);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream0 = 0b00001110 & 0b00001100;

// col 1 row 2 = green
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00010000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream1 = 0b00111000 & 0b00010000;

// col 1 row 3 = yellow
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b01100000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream2 = 0b11100000 & 0b01100000;

// third shift register goes unused in this column; but still
// included for consistency
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
bitstream3 = 0b00000000;

// latches high to send data
```

```
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// small delay to see colors
delay(100);

// checking to see if the random breakpoint is reached
if (i == breakpoint1){
  break;
}
// if not, add 1 to i
else{
  i++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// col 1 row 1 = blue
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00001000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream0 = 0b00001110 & 0b00001000;

// col 1 row 2 = blue-green
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00110000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream1 = 0b00111000 & 0b00110000;

// col 1 row 3 = green
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b01000000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream2 = 0b11100000 & 0b01000000;

// third shift register goes unused in this column; but still
// included for consistency
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
bitstream3 = 0b00000000;
```

```
// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
// small delay to see colors
delay(100);

// checking to see if the random breakpoint is reached
if (i == breakpoint1){
  break;
}
// if not, add 1 to i
else{
  i++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// col 1 row 1 = purple
shiftOut(dataPin0, clockPin0, MSBFIRST, 0b00001010);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream0 = 0b00001110 & 0b00001010;

// col 1 row 1 = blue
shiftOut(dataPin1, clockPin1, MSBFIRST, 0b00100000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream1 = 0b00111000 & 0b00100000;

// col 1 row 1 = blue-green
shiftOut(dataPin2, clockPin2, MSBFIRST, 0b11000000);
// storing the value of the led by grabbing only the three bits
// that control it
bitstream2 = 0b11100000 & 0b11000000;

// third shift register goes unused in this column; but still
// included for consistency
shiftOut(dataPin3, clockPin3, MSBFIRST, 0b00000000);
bitstream3 = 0b00000000;
```

```
  // latches high to send data
  digitalWrite(latchPin0, HIGH);
  digitalWrite(latchPin1, HIGH);
  digitalWrite(latchPin2, HIGH);
  digitalWrite(latchPin3, HIGH);
  // small delay to see colors
  delay(100);

  // checking to see if the random breakpoint is reached
  if (i == breakpoint1){
    break;
  }
  // if not, add 1 to i
  else{
    i++;
  }
  }
  delay(1000);


//-------------------------------------------------------
//-------------------------------------------------------


 // scrolling second column


//-------------------------------------------------------
//-------------------------------------------------------


  // setting a random breakpoint between 12 and 51 color changes
  \\ for the second while loop
  int breakpoint2 = random(12, 51);
  // start with 0 color changes
  int i_2 = 0;

  // making sure while loop does not exceed 51 color changes
  while(i_2 <51){
  // latches low to set data
  digitalWrite(latchPin0, LOW);
  digitalWrite(latchPin1, LOW);
  digitalWrite(latchPin2, LOW);
  digitalWrite(latchPin3, LOW);
```

```
// row 1 = red
// row 2 = purple
// row 3 = blue

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b00010000);
// holding the value of the first column led  and the second column led
bitstream00 = 0b01111110 & (bitstream0 | 0b00010000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b01000000);
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b01000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000010);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000010);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 |0b00001000);
// holding the value of the first column led  and the second column led
bitstream33 = 0b00001110 & (bitstream3 |0b00001000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_2 == breakpoint2){
  break;
}
// if not, add 1 to i
else{
  i_2++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = yellow
// row 2 = red
```

```
// row 3 = purple

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b00110000);
// holding the value of the first column led  and the second column led
bitstream00 = 0b01111110 & (bitstream0 | 0b00110000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b01000000);
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b01000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000000);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000000);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 | 0b00001010);
// holding the value of the first column led  and the second column led
bitstream33 = 0b00001110 & (bitstream3 | 0b00001010);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_2 == breakpoint2){
  break;
}
// if not, add 1 to i
else{
  i_2++;
}
// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = green
// row 2 = yellow
// row 3 = red

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b00100000);
// holding the value of the first column led  and the second column led
```

```
bitstream00 = 0b01111110 & (bitstream0 | 0b00100000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b11000000);
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b11000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000000);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000000);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 | 0b00000010);
// holding the value of the first column led  and the second column led
bitstream33 = 0b00001110 & (bitstream3 | 0b00000010);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_2 == breakpoint2){
  break;
}
// if not, add 1 to i
else{
  i_2++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = blue-green
// row 2 = green
// row 3 = yellow

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b01100000);
// holding the value of the first column led  and the second column led
bitstream00 = 0b01111110 & (bitstream0 | 0b01100000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b10000000);
```

```
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b10000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000000);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000000);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 | 0b00000110);
// holding the value of the first column led  and the second column led
bitstream33 = 0b00001110 & (bitstream3 | 0b00000110);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_2 == breakpoint2){
  break;
}
// if not, add 1 to i
else{
  i_2++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = blue
// row 2 = blue-green
// row 3 = green

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b01000000);
// holding the value of the first column led  and the second column led
bitstream00 = 0b01111110 & (bitstream0 | 0b01000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b10000000);
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b10000000);
```

```
shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000010);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000010);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 | 0b00000100);
// holding the value of the first column led  and the second column led
bitstream33 = 0b00001110 & (bitstream3 | 0b00000100);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_2 == breakpoint2){
  break;
}
// if not, add 1 to i
else{
  i_2++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = purple
// row 2 = blue
// row 3 = blue-green

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream0 | 0b01010000);
// holding the value of the first column led  and the second column led
bitstream00 = 0b01111110 & (bitstream0 | 0b01010000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream1 | 0b00000000);
// holding the value of the first column led  and the second column led
bitstream11 = 0b11111000 & (bitstream1 | 0b00000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream2 | 0b00000010);
// holding the value of the first column led  and the second column led
bitstream22 = 0b11100010 & (bitstream2 | 0b00000010);
```

```
  shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream3 | 0b00001100);
  // holding the value of the first column led  and the second column led
  bitstream33 = 0b00001110 & (bitstream3 | 0b00001100);

  // latches high to send data
  digitalWrite(latchPin0, HIGH);
  digitalWrite(latchPin1, HIGH);
  digitalWrite(latchPin2, HIGH);
  digitalWrite(latchPin3, HIGH);
  delay(100);

  // checking to see if the random breakpoint is reached
  if (i_2 == breakpoint2){
    break;
  }
  // if not, add 1 to i
  else{
    i_2++;
  }
  }

  delay(1000);


//-------------------------------------------------------------
//-------------------------------------------------------------


// scrolling column 3


//-------------------------------------------------------------
//-------------------------------------------------------------



// setting a random breakpoint between 12 and 51 color changes
// for the third while loop
int breakpoint3 = random(12, 51);
// setting i to 0 color changes
int i_3 = 0;

  // making sure while loop does not exceed 51 color changes
  while(i_3 <51){
```

```
// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = red
// row 2 = purple
// row 3 = blue

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b10000000);
// holding the value of the first, second, and third column leds
bitstream000 = 0b11111110 & (bitstream00 | 0b10000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000000);
// holding the value of the first, second, and third column leds
bitstream111 = 0b11111110 & (bitstream11 | 0b00000000);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00010100);
// holding the value of the first, second, and third column leds
bitstream222 = 0b11111110 & (bitstream22 | 0b00010100);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b01000000);
// holding the value of the first, second, and third column leds
bitstream333 = 0b01111110 & (bitstream33 | 0b01000000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_3 == breakpoint3){
  break;
}
// if not, add 1 to i
else{
  i_3++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
```

```
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = yellow
// row 2 = red
// row 3 = purple

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b10000000);
// holding the value of the first, second, and third column leds
bitstream000 = 0b11111110 & (bitstream00 | 0b10000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000010);
// holding the value of the first, second, and third column leds
bitstream111 = 0b11111110 & (bitstream11 | 0b00000010);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00000100);
// holding the value of the first, second, and third column leds
bitstream222 = 0b11111110 & (bitstream22 | 0b00000100);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b01010000);
// holding the value of the first, second, and third column leds
bitstream333 = 0b01111110 & (bitstream33 | 0b01010000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_3 == breakpoint3){
  break;
}
// if not, add 1 to i
else{
  i_3++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);
```

```
// row 1 = green
// row 2 = yellow
// row 3 = red

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b00000000);
// holding the value of the first, second, and third column leds
bitstream000 = 0b11111110 & (bitstream00 | 0b00000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000100);
// holding the value of the first, second, and third column leds
bitstream111 = 0b11111110 & (bitstream11 | 0b00000100);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00011000);
// holding the value of the first, second, and third column leds
bitstream222 = 0b11111110 & (bitstream22 | 0b00011000);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b00100000);
// holding the value of the first, second, and third column leds
bitstream333 = 0b01111110 & (bitstream33 | 0b00100000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_3 == breakpoint3){
  break;
}
// if not, add 1 to i
else{
  i_3++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = blue-green
// row 2 = green
// row 3 = yellow
```

```
shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b00000000);
// holding the value of the first, second, and third column leds
bitstream000 = 0b11111110 & (bitstream00 | 0b00000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000110);
// holding the value of the first, second, and third column leds
bitstream111 = 0b11111110 & (bitstream11 | 0b00000110);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00001000);
// holding the value of the first, second, and third column leds
bitstream222 = 0b11111110 & (bitstream22 | 0b00001000);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b00110000);
// holding the value of the first, second, and third column leds
bitstream333 = 0b01111110 & (bitstream33 | 0b00110000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_3 == breakpoint3){
  break;
}
// if not, add 1 to i
else{
  i_3++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = blue
// row 2 = blue-green
// row 3 = green

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b00000000);
// holding the value of the first, second, and third column leds
```

```
bitstream000 = 0b11111110 & (bitstream00 | 0b00000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000010);
// holding the value of the first, second, and third column leds
bitstream111 = 0b11111110 & (bitstream11 | 0b00000010);

shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00001100);
// holding the value of the first, second, and third column leds
bitstream222 = 0b11111110 & (bitstream22 | 0b00001100);

shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b00010000);
// holding the value of the first, second, and third column leds
bitstream333 = 0b01111110 & (bitstream33 | 0b00010000);

// latches high to send data
digitalWrite(latchPin0, HIGH);
digitalWrite(latchPin1, HIGH);
digitalWrite(latchPin2, HIGH);
digitalWrite(latchPin3, HIGH);
delay(100);

// checking to see if the random breakpoint is reached
if (i_3 == breakpoint3){
  break;
}
// if not, add 1 to i
else{
  i_3++;
}

// latches low to set data
digitalWrite(latchPin0, LOW);
digitalWrite(latchPin1, LOW);
digitalWrite(latchPin2, LOW);
digitalWrite(latchPin3, LOW);

// row 1 = purple
// row 2 = blue
// row 3 = blue-green

shiftOut(dataPin0, clockPin0, MSBFIRST, bitstream00 | 0b10000000);
// holding the value of the first, second, and third column leds
bitstream000 = 0b11111110 & (bitstream00 | 0b10000000);

shiftOut(dataPin1, clockPin1, MSBFIRST, bitstream11 | 0b00000100);
```

```
  // holding the value of the first, second, and third column leds
  bitstream111 = 0b11111110 & (bitstream11 | 0b00000100);

  shiftOut(dataPin2, clockPin2, MSBFIRST, bitstream22 | 0b00010000);
  // holding the value of the first, second, and third column leds
  bitstream222 = 0b11111110 & (bitstream22 | 0b00010000);

  shiftOut(dataPin3, clockPin3, MSBFIRST, bitstream33 | 0b01100000);
  // holding the value of the first, second, and third column leds
  bitstream333 = 0b01111110 & (bitstream33 | 0b01100000);

  // latches high to send data
  digitalWrite(latchPin0, HIGH);
  digitalWrite(latchPin1, HIGH);
  digitalWrite(latchPin2, HIGH);
  digitalWrite(latchPin3, HIGH);
  delay(100);

  // checking to see if the random breakpoint is reached
  if (i_3 == breakpoint3){
    break;
  }
  // if not, add 1 to i
  else{
    i_3++;
  }
  }


//-------------------------------------------------------
//-------------------------------------------------------


// win lose


//-------------------------------------------------------
//-------------------------------------------------------


// grabbing the bits from all leds in row 3
// moving the led bits over to make them comparable
byte bitsA = (bitstream2 & 0b11100000) >> 5;
// this number at the end is essentially deleting the zeros
byte bitsB = (bitstream33 & 0b00001110) >> 1;
```

```
byte bitsC = (bitstream333 & 0b01110000) >> 4;

// printing for troubleshooting
Serial.print("bitsA: "); Serial.println(bitsA, BIN);
Serial.print("bitsB: "); Serial.println(bitsB, BIN);
Serial.print("bitsC: "); Serial.println(bitsC, BIN);

// if all 3 leds are equal
if (bitsA == bitsB && bitsB == bitsC) {
  // blinking green on medium led
  digitalWrite(A1, HIGH);
  delay(200);
  digitalWrite(A1, LOW);
  delay(200);
  digitalWrite(A1, HIGH);
  delay(200);
  digitalWrite(A1, LOW);
  delay(200);
  digitalWrite(A1, HIGH);
  delay(200);
  digitalWrite(A1, LOW);
  delay(200);
  digitalWrite(A1, HIGH);
  delay(200);
  digitalWrite(A1, LOW);
  delay(200);
  digitalWrite(A1, HIGH);
  delay(200);
  digitalWrite(A1, LOW);
  delay(200);
  digitalWrite(A1, HIGH);
  // hold green for 10 s
  delay(5000);
  // led off
  digitalWrite(A1, LOW);
}
// if leds are different
else {
  // medium led turns red
  digitalWrite(A0, HIGH);
  // holds for 5 s
  delay(3000);
  // led off
  digitalWrite(A0, LOW);
}
```

```
// reset game
scrollStarted = false;
}
}
```